

Physics-Informed Neural Network

for Beam Parameter Identification

Project Documentation

Abstract

A reference document for understanding and navigating the project. Written assuming a data-science background — neural networks, gradient descent, autograd, training loops are all familiar — and minimal physics background. Physics concepts are introduced as they appear.

Contents

1	What the project is doing	3
1.1	The goal	3
1.2	The data	3
1.3	Why a PINN	3
2	The physics, in data-science terms	3
2.1	The beam, the FRF, and what's being measured	3
2.2	The governing equation	4
2.3	Boundary conditions	4
2.4	Complex modulus and damping	5
2.5	The displacement-to-velocity relationship	5
3	The data	5
3.1	Source file	5
3.2	Characteristics	5
3.3	Frequency cutoff decision	6
4	PINN architecture	6
4.1	WNet — displacement field	6
4.2	ENet — spatially varying stiffness	6
4.3	Scalar trainable parameters	7
4.4	How everything connects	7
5	Loss function	7
5.1	L_{PDE} — physics residual	7
5.2	L_{BC} — boundary residuals	8
5.3	L_{data} — fit to measurements	8
5.4	Why these specific weights	8

6 Collocation points	8
7 Training	9
7.1 Two phases	9
7.2 Cosine annealing with warm restarts	9
7.3 Float64 throughout	10
7.4 Checkpointing	10
8 Preprocessing pipeline evolution	10
8.1 v3 — initial spatially varying $E(x)$	10
8.2 v4 — Re/Im smoothing	10
8.3 v5 — narrower window + peak-aware sampling	11
8.4 v6 — no training-data smoothing, detection-only smoothing	11
8.5 Why the supervisor’s “true PINN” concern matters here	11
9 Code implementation	12
9.1 Project organisation and setup	12
9.2 Geometry constants	12
9.3 Data loading: <code>load_frf</code>	13
9.4 Preprocessing (v6 plan)	14
9.5 Tensor conversion and normalisation	15
9.6 Network architectures	16
9.7 Trainable scalars with softplus reparameterisation	17
9.8 Collocation grid	17
9.9 The autograd helper	18
9.10 The loss function	18
9.11 Training loop (Phase 1: Adam)	20
9.12 Training loop (Phase 2: L-BFGS)	21
9.13 Checkpointing	21
9.14 Evaluation and outputs	22
9.15 End-to-end flow recap	23
10 Implementation details (lower-level)	24
10.1 Manual real/imaginary handling	24
10.2 Why fourth-order autograd is the bottleneck	24
10.3 Normalisation	24
11 Outputs and validation	24
11.1 Files produced	24
11.2 Plots produced during the notebook run	25
11.3 What “converged” looks like	25
11.4 When to suspect a problem	25
12 Open issues and next steps	26
12.1 Synthetic validation (supervisor concern)	26
12.2 Coherence- or SNR-weighted data loss (fine-tuning step)	26
12.3 Multi-experiment warm-starting	26
12.4 HPC for parallel experiment campaigns	26
12.5 Phase loss term	27
A Notebook cell map	27

1. What the project is doing

1.1 The goal

Recover five physical quantities of a metal beam from experimental vibration measurements. The five unknowns are listed in Table 1.

Symbol	What it is	Type
$E(x)$	Young's modulus along the beam — how stiff the beam is at each position x	Function of position
η	Loss factor — how much energy the beam dissipates as it vibrates	Scalar
k_x	Stiffness of the translational spring at the clamped end (how rigidly the beam is held in place)	Scalar
k_φ	Stiffness of the rotational spring at the clamped end (how rigidly the beam is held against rotation)	Scalar
F	Amplitude of the force exciting the beam at the free tip	Scalar

Table 1: The five unknown physical quantities.

The known quantities are beam length, width, thickness, density, and tip mass.

1.2 The data

A frequency response function (FRF) measured at the tip of the beam. This is a complex-valued function $V(\omega)$ of frequency ω — for each frequency at which you excite the beam, you get a complex number describing how the tip moves. Standard equipment output.

1.3 Why a PINN

Classical curve-fitting techniques can identify η , the spring stiffnesses, and force amplitude assuming E is a single scalar (a constant material stiffness). The novelty here is that E varies along the beam — $E(x)$ is a *function*, not a number. Classical methods don't handle this. A physics-informed neural network can represent $E(x)$ as a learned function while the physics constraints pin down its values.

In data-science terms: this is an **inverse problem with a function-valued unknown**. The network is essentially doing functional regression, but with the regression target shaped by a partial differential equation rather than a labelled dataset.

2. The physics, in data-science terms

2.1 The beam, the FRF, and what's being measured

The setup is a thin metal strip clamped at one end (like a diving board), with a small mass attached at the free end. The beam is shaken at the free end with a sinusoidal force at various frequencies, and a sensor at the same location measures the resulting motion (specifically, the *velocity* at the tip).

At each excitation frequency, you measure the ratio of output (velocity) to input (force). This ratio, as a complex number, is the FRF value at that frequency. The complex part matters because output is typically phase-shifted from input — the beam doesn't respond instantaneously.

Sweep the frequency across a range and you get the FRF $V(\omega)$ — a function of frequency. Plot its magnitude in dB and you see **resonance peaks** at frequencies where the beam “wants” to vibrate (its natural frequencies, set by geometry and material). The amplitude at these peaks is huge; in between, the beam barely moves.

A useful analogy: an FRF is like the frequency-domain response of an audio system. Resonances are like the natural frequencies of a guitar string. Each peak is a “mode” — a particular pattern of beam deflection that vibrates coherently.

2.2 The governing equation

The physics is described by the Euler–Bernoulli beam equation. For a beam with stiffness $E(x)$ that varies along its length:

$$\frac{\partial^2}{\partial x^2} \left[E(x) \cdot I \cdot \frac{\partial^2 W}{\partial x^2} \right] - \rho \cdot A \cdot \omega^2 \cdot W = 0 \quad (1)$$

Reading the symbols:

- $W(x, \omega)$: complex displacement at position x , frequency ω .
- $E(x) \cdot I$: bending stiffness (I is a known geometric constant).
- $\rho \cdot A$: mass per unit length (known).
- $\omega^2 \cdot W$: inertial term (acceleration is $-\omega^2 \cdot W$ in frequency domain).

In words: at every point along the beam and every excitation frequency, the bending force balances the inertial force. **For the equation to be satisfied, the right $E(x)$ must be in place.** That’s how the PINN identifies $E(x)$: only the correct $E(x)$ makes the network’s predicted W satisfy this equation everywhere.

Expanding the second derivative of a product (the chain rule applied twice) gives the form actually computed in code:

$$E \cdot W'''' + 2 \cdot E' \cdot W''' + E'' \cdot W'' = \frac{\rho A}{I} \cdot \omega^2 \cdot W \quad (2)$$

This has *fourth-order* spatial derivatives of W , which is what makes the PyTorch autograd chain expensive (more on this in Section 10).

2.3 Boundary conditions

The beam is clamped at $x = 0$ and free at $x = L$. In idealised form, “clamped” means zero displacement and zero slope at the clamp; “free” means zero bending moment and zero shear force at the tip. But real clamps aren’t perfectly rigid — they’re modelled with two springs:

- A **translational spring** (k_x) that resists displacement at the clamp.
- A **rotational spring** (k_φ) that resists rotation at the clamp.

At the tip, there’s a known mass and an unknown excitation force F . So the boundary conditions are:

Location	Condition	Physical meaning
$x = 0$	$E \cdot I \cdot W''(0) + k_\varphi \cdot W'(0) = 0$	Bending moment balanced by rotational spring
$x = 0$	$E \cdot I \cdot W'''(0) + k_x \cdot W(0) = 0$	Shear force balanced by translational spring
$x = L$	$E \cdot I \cdot W''(L) = 0$	Zero bending moment at free tip
$x = L$	$E \cdot I \cdot W'''(L) + m_{\text{tip}} \cdot \omega^2 \cdot W(L) + F = 0$	Shear force balanced by tip inertia and applied force

Table 2: Boundary conditions enforced as loss terms.

These are enforced as additional loss terms (boundary residuals).

2.4 Complex modulus and damping

Real metals dissipate energy when they vibrate — they’re not perfectly elastic. This is modelled by making the stiffness complex:

$$E^*(x) = E(x) \cdot (1 + j\eta) \quad (3)$$

The real part $E(x)$ is the stiffness; the imaginary part $\eta \cdot E(x)$ is the damping. η (typically ~ 0.001 to 0.05 for metals) is what determines how sharp the resonance peaks are — small η means tall, narrow peaks; large η means short, wide peaks.

In code, this is handled by treating real and imaginary parts of every quantity separately and propagating them through the equations manually (see Section 10).

2.5 The displacement-to-velocity relationship

The measurement is velocity, but the natural variable for the PDE is displacement. They’re related by:

$$V(x, \omega) = j\omega \cdot W(x, \omega) \quad (4)$$

(Velocity is the time derivative of displacement; in frequency domain, taking a time derivative multiplies by $j\omega$.)

So the network predicts displacement $W = W_r + j \cdot W_i$, and the velocity FRF used for the data loss is computed by:

$$V_{\text{real}} = -\omega \cdot W_i \quad (5)$$

$$V_{\text{imag}} = +\omega \cdot W_r \quad (6)$$

(Multiplying $W_r + j \cdot W_i$ by $j\omega$ gives $-\omega \cdot W_i + j \cdot \omega \cdot W_r$.)

3. The data

3.1 Source file

Each measurement is saved as a `.numpy` dict file with the keys shown in Table 3.

Key	Type	What it is
<code>t</code>	float64 array	Time-domain samples — what the sensor recorded
<code>s1_t</code>	float64 array	Velocity time history at the measurement point
<code>freq_s1</code>	float64 array	Frequencies (Hz) of the FRF
<code>s1_fft</code>	complex128 array	The FRF itself — complex values, one per frequency

Table 3: Contents of the source `.numpy` file.

The pipeline uses `freq_s1` and `s1_fft`. The time-domain arrays are unused by current preprocessing.

3.2 Characteristics

For the reference dataset (`Output.numpy`):

- 96,000 frequency points, spaced 0.25 Hz apart.
- Frequency range: 0 to $\sim 24,000$ Hz.
- Complex velocity FRF.

- 7 clear resonance peaks visible up to ~ 5500 Hz (roughly at 50, 290, 810, 1180, 1620, 2640, 4080, and one weaker mode near 5500 Hz).
- Noise floor at ~ -110 dB, peak heights from -20 dB (first mode) to -77 dB (highest usable mode).
- Above ~ 6 kHz, signal merges into the noise floor and no further modes are reliably resolved.

3.3 Frequency cutoff decision

Hard cutoff at 5000–5500 Hz. The reasons aren't physical (Euler–Bernoulli theory remains valid above this) — they're empirical:

- Above this point, the FRF is pure noise, with no identifiable resonances.
- Training on noise wastes optimizer effort and can corrupt parameter estimates.

Cutting lower (e.g., at 2500 Hz) would discard modes 5–7, which carry most of the spatial-variation information about $E(x)$. Not worth doing.

4. PINN architecture

Two neural networks plus four scalar trainable parameters.

4.1 WNet — displacement field

Purpose: predict the displacement field $W(x, \omega)$ at any beam position and excitation frequency.

Input: (x, ω) — two real numbers
 Output: $(W_{\text{real}}, W_{\text{imag}})$ — two real numbers (the complex displacement)
 Hidden: 4 layers $\times$ 128 units, tanh activations
 Params: $\sim 50,000$

Inputs are normalised to $[-1, 1]$ before being fed in (the network sees position scaled by beam length and frequency scaled by the training range). Outputs are unnormalised real numbers; they directly represent displacement in physical units (metres).

This is a standard MLP. Nothing exotic. The physics is enforced through the loss function, not the architecture.

4.2 ENet — spatially varying stiffness

Purpose: predict the Young's modulus $E(x)$ at any beam position.

Input: x — one real number
 Output: $E(x)$ — one real number, bounded to $[50, 400]$ GPa
 Hidden: 3 layers $\times$ 64 units, tanh activations
 Params: $\sim 8,500$

The output goes through a sigmoid that rescales it to a fixed physical range (50–400 GPa, covering soft metals to ceramics). This bounding was added in v4 because earlier versions had $E(x)$ exploding to nonphysical values (>4000 GPa). The last layer is initialised so that the sigmoid output starts at the user-provided `E0_init` (~ 200 GPa for steel).

$E(x)$ is purely real. The complex modulus $E^*(x) = E(x) \cdot (1 + j\eta)$ is built afterwards by combining $E(x)$ from this network with the scalar η .

4.3 Scalar trainable parameters

Four physical scalars, each implemented as a `torch.nn.Parameter` with a softplus reparameterisation to enforce positivity:

$$\eta = \text{softplus}(\eta_{\text{raw}}) \cdot \eta_{\text{scale}} \quad (7)$$

$$k_x = \text{softplus}(k_{x,\text{raw}}) \cdot k_{x,\text{scale}} \quad (8)$$

$$k_\varphi = \text{softplus}(k_{\varphi,\text{raw}}) \cdot k_{\varphi,\text{scale}} \quad (9)$$

$$F = \text{softplus}(F_{\text{raw}}) \cdot F_{\text{scale}} \quad (10)$$

The `_raw` versions are what Adam actually updates (unbounded reals); softplus produces the physically meaningful positive quantities. The `_scale` constants set sensible orders of magnitude ($k_{x,\text{scale}} = 10^3$ N/m, etc.) so that all `_raw` variables stay in roughly the same numerical range.

4.4 How everything connects

There is **one Adam optimizer**, not separate ones for each network. All gradients are computed in a single backward pass and all parameters updated in a single step:

1. `WNet`(x, ω) $\rightarrow (W_r, W_i)$
2. `ENet`(x) $\rightarrow E(x)$
3. $\eta, k_x, k_\varphi, F = \text{softplus}(\dots)$
4. `autograd` $\rightarrow W', W'', W''', W''''$
5. `autograd` $\rightarrow E', E''$
6. Assemble loss (PDE + BC + data)
7. `loss.backward()` — gradients flow back into every trainable parameter simultaneously.

5. Loss function

Three components, summed with fixed weights:

$$L_{\text{total}} = w_{\text{PDE}} \cdot L_{\text{PDE}} + w_{\text{BC}} \cdot L_{\text{BC}} + w_{\text{data}} \cdot L_{\text{data}} \quad (11)$$

with $w_{\text{PDE}} = 10$, $w_{\text{BC}} = 100$, $w_{\text{data}} = 1$.

5.1 L_{PDE} — physics residual

Computed on a grid of collocation points spanning the (x, ω) domain. At each collocation point, we plug the network's predictions into the beam equation and measure how far from zero the result is:

$$\text{residual}_{\text{real}} = I \cdot (\mathcal{L}W_r - \eta \cdot \mathcal{L}W_i) - \rho \cdot A \cdot \omega^2 \cdot W_r \quad (12)$$

$$\text{residual}_{\text{imag}} = I \cdot (\mathcal{L}W_i + \eta \cdot \mathcal{L}W_r) - \rho \cdot A \cdot \omega^2 \cdot W_i \quad (13)$$

$$\text{where } \mathcal{L}W = E \cdot W'''' + 2 \cdot E' \cdot W''' + E'' \cdot W'' \quad (14)$$

$$L_{\text{PDE}} = \text{mean}(\text{residual}_{\text{real}}^2 + \text{residual}_{\text{imag}}^2) \quad (15)$$

This is the term that constrains $E(x)$. The PDE has to hold everywhere along the beam, not just at x_{meas} , so the optimizer is forced to find an $E(x)$ profile that makes the predicted $W(x, \omega)$ physically consistent at every spatial location.

5.2 L_{BC} — boundary residuals

Evaluated at $x = 0$ (clamped end with springs) and $x = L$ (free tip with mass and force). Same idea — plug predictions into the boundary equations, square the residuals, average them. This is what constrains k_x , k_φ , m_{tip} , and F . Weight is 100 (highest) because boundary conditions are exact constraints — they must hold precisely.

5.3 L_{data} — fit to measurements

Compares the network’s predicted FRF at the measurement location $x = L$ to the measured FRF, at the training frequencies:

$$L_{\text{data}} = \text{mean} \left[(V_{\text{pred}}^{\text{real}} - V_{\text{meas}}^{\text{real}})^2 + (V_{\text{pred}}^{\text{imag}} - V_{\text{meas}}^{\text{imag}})^2 \right] \quad (16)$$

$V_{\text{pred}} = j\omega \cdot W$ is computed from the network’s W output as described in Section 2. Weight is 1 because the data is trusted but contains measurement noise.

This is the term that pins down η : the heights of the resonance peaks scale with $1/\eta$, so getting the peak heights right pins down η .

5.4 Why these specific weights

The relative weighting $(w_{\text{PDE}}, w_{\text{BC}}, w_{\text{data}}) = (10, 100, 1)$ reflects a priority ordering:

- **BC most important:** boundary conditions are exact physical statements — any violation is a real physical inconsistency.
- **PDE second:** the equation must hold, but residual numerical noise is acceptable.
- **Data least:** measurements have noise; over-fitting them means fitting noise.

If your loss landscape ever becomes pathological — e.g., physics terms dominate and the network ignores the data — try $(1, 10, 1)$ or $(5, 50, 1)$ to rebalance.

6. Collocation points

Collocation points are different from training points. This is a common source of confusion.

	Training points	Collocation points
Where used	Data loss L_{data}	PDE & BC losses
What’s compared	V_{pred} vs V_{meas}	PDE residual vs zero
Need measurements?	Yes — frequencies must match experiment	No — anywhere in (x, ω) domain
Count	~ 500 after peak-aware sampling	2,500 (50×50 grid)
Dimension	1D in frequency (fixed $x = x_{\text{meas}}$)	2D in (x, ω)

Table 4: Training points vs collocation points.

The collocation grid is $N_x \times N_\omega = 50 \times 50$ uniformly distributed:

- $x \in [0, L]$ (sample many positions along the beam).
- $\omega \in [\omega_{\text{min}}, \omega_{\text{max}}]$ (span the training frequency range).

Why this matters for $E(x)$ identification. At the measurement point $x = x_{\text{meas}}$, the FRF only constrains $W(x_{\text{meas}}, \omega)$ — a 1D slice. To identify $E(x)$ at *other* locations, you need physics constraints at those locations. That’s what the collocation grid does: it asks “does the predicted

displacement satisfy the PDE at every interior position, including those without measurements?” The PDE involves $E(x)$, so demanding the PDE hold at $x = 0.5L$ forces $E(0.5L)$ to be self-consistent with the predicted W there.

In data-science terms: the collocation grid extends the regression problem from “fit V_{meas} at one location” to “find the function $E(x)$ that makes the entire physics consistent everywhere”. Without it, $E(x)$ away from x_{meas} would be totally unconstrained.

7. Training

7.1 Two phases

Phase 1: Adam, 20,000 epochs

- Cosine annealing with warm restarts ($T_0 = 4000$, $T_{\text{mult}} = 2$).
- Initial learning rate 10^{-3} for all parameters.
- Checkpoints to Google Drive every 1000 epochs.

Phase 2: L-BFGS, 300 steps

- `max_iter=20` per step (was 50 in v3, reduced for speed).
- Strong Wolfe line search.
- Checkpoints every 30 steps.

Why two phases. Adam is good for broad exploration of complex loss landscapes — it can escape local minima and make rapid initial progress. But near the minimum, Adam’s stochastic updates are inefficient. L-BFGS uses curvature information (second-order Hessian approximation) and converges much faster than first-order methods when you’re close to a minimum.

Standard pattern in PINN literature: Adam for the first $\sim 80\text{--}90\%$ of training, then L-BFGS for final polish.

7.2 Cosine annealing with warm restarts

A learning rate schedule that:

1. Starts at the initial LR.
2. Decays smoothly (cosine curve) to a minimum.
3. Suddenly resets back to the initial LR.
4. Repeats with a longer period each time.

Concretely, with $T_0 = 4000$, $T_{\text{mult}} = 2$:

- Epochs 0–4000: LR decays from 10^{-3} to 10^{-6} .
- Epoch 4000: LR resets to 10^{-3} .
- Epochs 4000–12000: LR decays again (cycle of 8000).
- Epoch 12000: another reset.
- Epochs 12000–28000: LR decays (cycle of 16000).

This was introduced in v4 to replace `ReduceLROnPlateau`, which was causing periodic loss spikes that destabilised training. The warm restart pattern is much smoother because it doesn't react to noise in the loss signal.

7.3 Float64 throughout

The PDE residual computes the *fourth* spatial derivative of the network's output (W'''') via four nested autograd calls. Each gradient computation accumulates floating-point rounding error. In float32 (single precision), the cumulative error in W'''' overwhelms the signal — the PDE residual is dominated by numerical noise, not physics.

Float64 (double precision) makes the rounding error 10^9 times smaller, which is enough to keep the fourth-order derivatives clean.

The practical consequence: training runs on CPU (or GPU with full float64 support). Apple Silicon's MPS backend doesn't support float64 well, so M-series Macs run this on CPU. Colab T4 GPUs do support float64 but with reduced throughput compared to float32. Training time is similar between a T4 and a modern CPU.

7.4 Checkpointing

State is saved to Google Drive periodically during both phases:

- Adam: every 1000 epochs (~20 saves over the full 20k-epoch phase).
- L-BFGS: every 30 steps.

Each checkpoint contains: both network state dicts, all four physical parameters, optimizer state, scheduler state, and the loss history. The training cells detect existing checkpoints and resume automatically. This is what makes long Colab runs reliable — if the session disconnects, you re-run the cell and pick up where you left off.

When training completes, a single `training_complete.pt` file is saved with everything (model state + history), and the intermediate checkpoints are deleted.

8. Preprocessing pipeline evolution

This is the most-revised part of the project. Each version addressed a specific failure in the previous one.

8.1 v3 — initial spatially varying $E(x)$

Smoothing: Savitzky–Golay filter on magnitude and phase separately.

Issue: Phase is wrapped to $[-\pi, \pi]$ and has discontinuities at every resonance and anti-resonance. SG-smoothing wrapped phase averages discontinuities together, producing garbage. The imaginary component of the training data was effectively destroyed.

8.2 v4 — Re/Im smoothing

Change: Smooth $\text{Re}(V)$ and $\text{Im}(V)$ separately instead of magnitude and phase. Both Re and Im are smooth continuous functions with no wrapping, so SG works on them.

Also added: $E(x)$ sigmoid bounding to $[50, 400]$ GPa; cosine annealing LR schedule replaced `ReduceLROnPlateau`; higher LR for scalar parameters.

SG window: 51 samples (computed dynamically as $\min(51, \text{len}/20)$).

Subsampling: 300 frequencies, uniform spacing.

Issues:

- Window $51 \times \text{spacing } 0.17 \text{ Hz} \approx 8.5 \text{ Hz}$ smoothing bandwidth. Resonance peaks are narrower than this, so SG flattens them dramatically.
- Uniform 300-point subsampling misses peaks: at 16.7 Hz spacing, nearest sample to a 1 Hz-wide peak might be 8 Hz off-centre, sitting on the side of the resonance rather than the top.
- These two effects combine to attenuate peaks by 30–40 dB in the training data, leading the optimizer to find η too large and/or F too small.

8.3 v5 — narrower window + peak-aware sampling

Change: SG window reduced to 15 samples ($\sim 2.5 \text{ Hz}$). Peak detection added via `scipy.signal.find_peaks`. Training point schedule shifted from uniform to peak-aware: ~ 50 points around each peak ($\pm 5 \cdot \text{FWHM}$), ~ 8 around each anti-resonance, ~ 100 sparse background.

Issues:

- Even window 15 was still too wide for the narrowest peaks (mode 1 has FWHM $\sim 0.25 \text{ Hz}$ at typical η values).
- Peak detection on the smoothed signal with `prominence=10 dB` picked up dozens of false peaks in noise-floor regions, producing 7000+ training points instead of ~ 500 .
- Smoothing Re and Im separately has a geometric issue near resonances: at the peak, one component has a sign change (its samples cancel under averaging) and the other has a localised spike (gets diluted by surrounding small values). Magnitude drops more under Re/Im smoothing than under direct magnitude smoothing.

8.4 v6 — no training-data smoothing, detection-only smoothing

The current plan. Key changes:

1. **Don't smooth the training data at all.** Peaks are too narrow to survive any meaningful smoothing. Use the raw V_{cut} directly.
2. **Smooth $|V|$ (magnitude in dB) separately, only for peak detection.** Magnitude is always positive — no sign-cancellation problem. Use a heavy smoothing (window=101) since we only need correct peak locations, not heights.
3. **Higher prominence threshold (20 dB)** to reject noise-floor false peaks.
4. **Larger distance parameter** (200 samples = 50 Hz) to enforce minimum spacing between detected peaks.

Why no smoothing on training data works: With ~ 40 training points per peak in regions of high SNR, random measurement noise on individual points partially cancels in the data loss sum. Stochastic noise averages out; the optimizer finds the noiseless underlying signal. This is preferable to smoothing, which would introduce *bias* (peak flattening) that doesn't average out.

8.5 Why the supervisor's "true PINN" concern matters here

Modal-model curve fitting (e.g., Rational Fraction Polynomial, RFP) is a classical method that fits the FRF with a sum of pole-residue terms, extracting modal frequencies, damping, and residues directly. If you used RFP to reconstruct a clean FRF and then fed that into the PINN, you'd be

Version	Training-data smoothing	Peak detection	Sampling
v3	SG on magnitude & phase	None	Uniform 300
v4	SG on Re & Im, window 51	None	Uniform 300
v5	SG on Re & Im, window 15	On smoothed signal, prom=10 dB	Peak-aware (~7000 pts: too many)
v6	None	On heavily-smoothed $ V $ (window 101), prom=20 dB	Peak-aware (~500 pts: target)

Table 5: Preprocessing pipeline versions.

smuggling the answers into the training data — RFP already extracts most of what the PINN is supposed to identify.

The current v6 pipeline avoids this: it uses raw experimental data (no curve fitting, no reconstructed signals) with smart sampling. RFP can still be useful as an *external* sanity check — fit it separately, compare its predicted FRF to the PINN’s, look at agreement — but it should never feed into training.

9. Code implementation

This section walks through the code structure with annotated snippets. The notebook is divided into logical blocks; each subsection below describes one block, the choices made, and points to watch.

9.1 Project organisation and setup

The notebook opens with imports, device configuration, and checkpoint helpers:

```

1 import os, io, json
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from scipy.signal import savgol_filter
6
7 import torch
8 import torch.nn as nn
9
10 DTYPE = torch.float64
11 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
12
13 torch.manual_seed(42)
14 np.random.seed(42)

```

Listing 1: Imports and device setup

Two non-obvious choices:

- `DTYPE = torch.float64`: as discussed in Section 7, float32 is too imprecise for fourth-order autograd. This must be set globally and used everywhere.
- Fixed random seeds: ensures runs are reproducible. With autograd-driven training, even small numerical perturbations can lead to different convergence paths.

9.2 Geometry constants

All known physical quantities are declared in one place:

```

1  # Beam geometry (fixed, known from measurement)
2  L          = 126.06e-3    # beam length [m]
3  width      = 10.13e-3    # width [m]
4  height     = 0.95e-3     # thickness [m]
5  rho        = 8216.0      # density [kg/m^3]
6  bending_axis = "weak"
7  x_meas     = L           # FRF measured at tip (free end)
8  m_tip      = 10e-3       # tip mass [kg]
9
10 # Frequency range
11 MIN_FREQ_HZ = 1.0        # avoid singular matrix at 0 Hz
12 MAX_FREQ_HZ = 5000.0     # hard cutoff
13
14 # Initial guesses for scalar unknowns
15 E0_init     = 200e9       # [Pa] initial guess for E(x) mean level
16 eta_init    = 0.01       # [-] loss factor
17 kx_init     = 1e5         # [N/m] translational spring at x=0
18 kphi_init   = 1e3        # [N*m/rad] rotational spring at x=0
19 F_init      = 0.1        # [N] tip force amplitude
20
21 # Derived geometry
22 A = width * height
23 if bending_axis.lower() == "weak":
24     I = width * height**3 / 12.0
25 else:
26     I = height * width**3 / 12.0

```

Listing 2: Beam geometry and initial parameter guesses

The initial guess `E0_init` matters: ENet's last layer is initialised so the sigmoid output starts at this value. Set it to the nominal stiffness of your beam's material before training.

9.3 Data loading: `load_frf`

A flexible loader that handles `.npz` dicts and `.npz` files with various key conventions:

```

1  def load_frf(path, x_meas):
2      ext = os.path.splitext(path)[1].lower()
3      fk  = ["freq", "f", "frequency", "f_hz", "freq_s1"]
4      vk  = ["frf", "V", "velocity", "response", "H", "s1_fft"]
5
6      if ext == ".npz":
7          data = np.load(path, allow_pickle=True)
8          fkey = _pick(data, fk); vkey = _pick(data, vk)
9          f = data[fkey].reshape(-1).astype(float)
10         V = data[vkey].reshape(-1)
11     elif ext == ".npy":
12         arr = np.load(path, allow_pickle=True)
13         data = arr.item() if hasattr(arr, "item") else None
14         if not isinstance(data, dict):
15             raise ValueError("Plain .npy unsupported -- use .npz")
16         fkey = _pick(data, fk); vkey = _pick(data, vk)
17         f = data[fkey].reshape(-1).astype(float)
18         V = data[vkey].reshape(-1)
19
20     if not np.iscomplexobj(V):
21         raise ValueError("FRF array must be complex")
22
23     # Clean up: remove NaN/Inf, enforce positive frequencies, sort
24     mask = np.isfinite(f) & np.isfinite(V.real) & np.isfinite(V.imag) & (f >=
25         0)
26     f, V = f[mask], V[mask]

```

```

26     order = np.argsort(f)
27     f, V = f[order], V[order]
28
29     # Average duplicate frequencies
30     df = pd.DataFrame({"f": f, "Vr": V.real, "Vi": V.imag}).groupby("f",
31                           as_index=False).mean()
32     f = df["f"].to_numpy()
33     V = df["Vr"].to_numpy() + 1j*df["Vi"].to_numpy()
34     x = np.full_like(f, x_meas)
35     return f, x, V

```

Listing 3: The data loading function

Key behaviours:

- Tries multiple known key names; many measurement systems save under different conventions.
- Rejects non-complex FRF arrays — the pipeline requires real-and-imaginary, not magnitude-and-phase.
- Cleans NaNs, sorts by frequency, averages any duplicate frequencies. Defensive against messy data files.

9.4 Preprocessing (v6 plan)

The v6 preprocessing block has three logical phases: cutoff, peak detection on a heavily-smoothed copy of $|V|$, and peak-aware sampling using the raw V values for training. Below is the planned implementation:

```

1  from scipy.signal import find_peaks, peak_widths
2
3  f_raw, x_raw, V_raw = load_frf(uploaded_name, x_meas=x_meas)
4
5  # 1. Hard frequency cutoff
6  mask = (f_raw >= MIN_FREQ_HZ) & (f_raw <= MAX_FREQ_HZ)
7  f_cut, V_cut = f_raw[mask], V_raw[mask]
8  df_raw = f_cut[1] - f_cut[0]
9
10 # 2. Peak detection on HEAVILY smoothed |V| in dB
11 # (used only for finding peak locations, never for training)
12 mag_db_raw = 20 * np.log10(np.abs(V_cut) + eps_db)
13 DET_WIN = 101
14 mag_db_detect = savgol_filter(mag_db_raw, window_length=DET_WIN, polyorder=3)
15
16 PEAK_PROMINENCE = 20.0 # dB above local noise floor
17 PEAK_DISTANCE = 200 # min separation in samples
18 peak_idx, _ = find_peaks(mag_db_detect,
19                           prominence=PEAK_PROMINENCE,
20                           distance=PEAK_DISTANCE)
21 peak_freqs = f_cut[peak_idx]
22
23 # 3. FWHM estimation (for adaptive band widths)
24 widths_samples, _, _ = peak_widths(mag_db_detect, peak_idx, rel_height=0.5)
25 peak_fwhm_hz = np.maximum(widths_samples * df_raw, 0.5)
26
27 # Anti-resonance detection: invert the signal and run find_peaks
28 antires_idx, _ = find_peaks(-mag_db_detect,
29                              prominence=PEAK_PROMINENCE,
30                              distance=PEAK_DISTANCE)
31 antires_freqs = f_cut[antires_idx]
32
33 # 4. Build peak-aware training set

```

```

34 PTS_PER_PEAK      = 40
35 PTS_PER_ANTIRES   = 8
36 PTS_BACKGROUND    = 100
37 BAND_FWHMS        = 3.0    # band half-width = 3 FWHMs
38
39 # Clip band widths so narrow peaks get a minimum band, wide ones get a cap
40 band_halfwidths = np.clip(BAND_FWHMS * peak_fwhm_hz, 5.0, 80.0)
41
42 f_train_list = []
43 for fp, hw in zip(peak_freqs, band_halfwidths):
44     f_train_list.append(np.linspace(fp - hw, fp + hw, PTS_PER_PEAK))
45 for fa in antires_freqs:
46     f_train_list.append(np.linspace(fa - 3.0, fa + 3.0, PTS_PER_ANTIRES))
47 f_train_list.append(np.linspace(f_cut[0], f_cut[-1], PTS_BACKGROUND))
48
49 # 5. Snap to nearest available raw frequencies, deduplicate
50 f_train_request = np.unique(np.concatenate(f_train_list))
51 snap_idx = np.searchsorted(f_cut, f_train_request)
52 snap_idx = np.clip(snap_idx, 0, len(f_cut) - 1)
53 left = np.clip(snap_idx - 1, 0, len(f_cut) - 1)
54 choose_left = (np.abs(f_cut[left] - f_train_request)
55               < np.abs(f_cut[snap_idx] - f_train_request))
56 snap_idx = np.unique(np.where(choose_left, left, snap_idx))
57
58 f_train = f_cut[snap_idx]
59 V_train = V_cut[snap_idx]    # raw, no smoothing
60 x_train = np.full_like(f_train, x_meas)

```

Listing 4: Preprocessing: cutoff, peak detection, peak-aware sampling

Subtle design points worth noting:

- `mag_db_detect` is built once and used for all three tasks (peak finding, FWHM estimation, anti-resonance finding). Smoothing magnitude (positive, no sign changes) preserves peak locations well; smoothing Re/Im separately would not.
- Training data is `V_cut[snap_idx]` — the *raw* FRF at peak-aware frequencies. No filtering touches the values used for training.
- The snap-to-nearest step guarantees `f_train` is a subset of `f_cut`, so the training data are real measured samples, not interpolated.

9.5 Tensor conversion and normalisation

After preprocessing, the NumPy arrays become PyTorch tensors:

```

1 omega_train = 2.0 * np.pi * f_train
2
3 x_t      = torch.tensor(x_train,      dtype=DTYPE, device=device).view(-1, 1)
4 omega_t  = torch.tensor(omega_train,  dtype=DTYPE, device=device).view(-1, 1)
5 Vr_t     = torch.tensor(V_train.real, dtype=DTYPE, device=device).view(-1, 1)
6 Vi_t     = torch.tensor(V_train.imag, dtype=DTYPE, device=device).view(-1, 1)
7
8 omega_min = float(omega_train.min())
9 omega_max = float(omega_train.max())
10
11 def norm_x(x):
12     return 2.0 * x / L - 1.0
13
14 def norm_omega(w):
15     return 2.0 * (w - omega_min) / (omega_max - omega_min) - 1.0

```

Listing 5: Tensor conversion and input normalisation

Why normalise: $\tanh$ layers expect inputs roughly in $[-1, 1]$. If you feed in x in metres (order 10^{-1}) or ω in rad/s (order 10^4), the first layer’s pre-activations saturate or vanish. Normalisation makes the loss landscape well-conditioned.

9.6 Network architectures

WNet (the displacement field) is a straightforward MLP with $\tanh$ activations:

```

1  class WNet(nn.Module):
2      """Displacement field: (x, omega) -> (W_real, W_imag)"""
3      def __init__(self, hidden=[2, 128, 128, 128, 128, 2]):
4          super().__init__()
5          self.layers = nn.ModuleList([
6              nn.Linear(hidden[i], hidden[i+1])
7              for i in range(len(hidden)-1)
8          ])
9          self.act = nn.Tanh()
10
11     def forward(self, x, omega):
12         z = torch.cat([norm_x(x), norm_omega(omega)], dim=1)
13         for layer in self.layers[:-1]:
14             z = self.act(layer(z))
15         out = self.layers[-1](z)
16         return out[:, 0:1], out[:, 1:2]    # Wr, Wi

```

Listing 6: WNet — displacement field network

ENet (Young’s modulus) has the sigmoid bounding logic and a custom initialisation:

```

1  E_MIN = 50e9      # 50 GPa  -- softest plausible metal
2  E_MAX = 400e9     # 400 GPa -- hardest plausible metal/ceramic
3
4  class ENet(nn.Module):
5      def __init__(self):
6          super().__init__()
7          self.net = nn.Sequential(
8              nn.Linear(1, 64), nn.Tanh(),
9              nn.Linear(64, 64), nn.Tanh(),
10             nn.Linear(64, 64), nn.Tanh(),
11             nn.Linear(64, 1),
12         )
13
14         # Initialize last layer so sigmoid(raw) approx E0_init
15         with torch.no_grad():
16             target_sigmoid = (E0_init - E_MIN) / (E_MAX - E_MIN)
17             init_bias = np.log(target_sigmoid / (1.0 - target_sigmoid))
18             self.net[-1].bias.fill_(init_bias)
19             self.net[-1].weight.fill_(0.0)
20
21     def forward(self, x):
22         xn = norm_x(x)
23         raw = self.net(xn)
24         return E_MIN + (E_MAX - E_MIN) * torch.sigmoid(raw)

```

Listing 7: ENet — bounded Young’s modulus network

The initialisation trick: the last layer’s weights are zero and its bias is set to $\logit((E_0\text{init} -$

$E_{\text{MIN}})/(E_{\text{MAX}} - E_{\text{MIN}}))$. With zero weights, the layer's output equals the bias regardless of input. After the sigmoid, this gives an initial $E(x) \approx E0_{\text{init}}$ uniformly across x . The network then learns the deviations from this prior during training. This avoids cold-start training instabilities.

9.7 Trainable scalars with softplus reparameterisation

```

1  def inv_sp(y):
2      """Inverse softplus: returns x such that softplus(x) = y."""
3      y = np.float64(y)
4      return float(np.where(y > 50, y, np.log(np.exp1(y))))
5
6  # Scale factors set sensible orders of magnitude for the raw variables
7  eta_scale = 1.0
8  kx_scale = 1e3      # N/m
9  kphi_scale = 1e1     # N*m/rad
10 F_scale = 1.0      # N
11
12 eta_raw = nn.Parameter(torch.tensor([inv_sp(eta_init / eta_scale)],
13                                     dtype=DTYPE, device=device))
14 kx_raw = nn.Parameter(torch.tensor([inv_sp(kx_init / kx_scale)],
15                                     dtype=DTYPE, device=device))
16 kphi_raw = nn.Parameter(torch.tensor([inv_sp(kphi_init / kphi_scale)],
17                                     dtype=DTYPE, device=device))
18 F_raw = nn.Parameter(torch.tensor([inv_sp(F_init / F_scale)],
19                                     dtype=DTYPE, device=device))
20
21 def get_params():
22     sp = torch.nn.functional.softplus
23     return (
24         (sp(eta_raw) + 1e-12) * eta_scale,
25         (sp(kx_raw) + 1e-12) * kx_scale,
26         (sp(kphi_raw) + 1e-12) * kphi_scale,
27         (sp(F_raw) + 1e-12) * F_scale,
28     )

```

Listing 8: Scalar parameters with positivity enforcement

The pattern is: each physical scalar p is stored as an unbounded p_{raw} , and exposed via $p = \text{softplus}(p_{\text{raw}}) \cdot p_{\text{scale}}$. This guarantees positivity (softplus is always ≥ 0) without using any constrained optimiser. The $+10^{-12}$ floor avoids division-by-zero in the loss when the optimiser drives a parameter very close to zero.

The optimiser is constructed with parameter groups, all in one Adam call:

```

1  optimizer = torch.optim.Adam([
2      {"params": list(w_net.parameters()), "lr": 1e-3},
3      {"params": list(e_net.parameters()), "lr": 1e-3},
4      {"params": [eta_raw, kx_raw, kphi_raw, F_raw], "lr": 1e-3},
5  ])
6
7  scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
8      optimizer, T_0=4000, T_mult=2, eta_min=1e-6
9  )

```

Listing 9: Single Adam optimizer over all trainable parameters

9.8 Collocation grid

```

1  Nx, Nw = 50, 50      # 2500 collocation points total
2

```

```

3 xc = torch.linspace(0.0, L, Nx, dtype=DTYPE, device=device)
4 wc = torch.linspace(omega_min, omega_max, Nw, dtype=DTYPE, device=device)
5
6 xx, ww = torch.meshgrid(xc, wc, indexing="ij")
7 x_col = xx.reshape(-1, 1).clone().detach().requires_grad_(True)
8 omega_col = ww.reshape(-1, 1).clone().detach()

```

Listing 10: Building the collocation grid

The crucial detail is `requires_grad_(True)` on `x_col`: this lets autograd differentiate the network's output with respect to x later, when we compute W' , W'' , W''' , W'''' . The ω tensor doesn't need gradients because the PDE only contains spatial derivatives.

9.9 The autograd helper

```

1 def grad(y, x):
2     return torch.autograd.grad(
3         y, x,
4         grad_outputs=torch.ones_like(y),
5         create_graph=True,      # keep the graph for further gradients
6         retain_graph=True,      # don't free it between calls
7     )[0]

```

Listing 11: Helper for taking gradients of gradients

`create_graph=True` is what enables *gradient of gradient* computations. Each call adds a new layer to the computational graph; chaining four `grad` calls gives you fourth-order derivatives. The graph size grows multiplicatively with derivative order, which is why fourth-order autograd is the dominant cost per training step.

9.10 The loss function

This is the heart of the project — the function that ties everything together. It computes all three loss components and the residuals at every point in the collocation grid:

```

1 def pin_loss(w_pde=10.0, w_bc=100.0, w_data=1.0):
2     eta, kx, kphi, F = get_params()
3
4     # E(x) and its derivatives at collocation points
5     E_col = e_net(x_col)
6     E_x = grad(E_col, x_col)
7     E_xx = grad(E_x, x_col)
8
9     # Displacement field and its spatial derivatives up to 4th order
10    Wr, Wi = w_net(x_col, omega_col)
11    Wr_x = grad(Wr, x_col); Wi_x = grad(Wi, x_col)
12    Wr_xx = grad(Wr_x, x_col); Wi_xx = grad(Wi_x, x_col)
13    Wr_xxx = grad(Wr_xx, x_col); Wi_xxx = grad(Wi_xx, x_col)
14    Wr_xxxx = grad(Wr_xxx, x_col); Wi_xxxx = grad(Wi_xxx, x_col)
15
16    omega2 = omega_col ** 2
17
18    # PDE residual: L[W] = E*W'''' + 2*E'*W''' + E''*W''
19    LW_r = E_col*Wr_xxxx + 2*E_x*Wr_xxx + E_xx*Wr_xx
20    LW_i = E_col*Wi_xxxx + 2*E_x*Wi_xxx + E_xx*Wi_xx
21
22    # Combine with damping: complex residual = I*(LW - i*eta*LW) - rho*A*omega
23    r_pde_r = I_t*(LW_r - eta*LW_i) - rho_t*A_t*omega2*Wr
24    r_pde_i = I_t*(LW_i + eta*LW_r) - rho_t*A_t*omega2*Wi

```

```

25 pde_loss = torch.mean(r_pde_r**2 + r_pde_i**2)
26 # ...

```

Listing 12: PINN loss: PDE residual at collocation points

The body continues with the boundary-condition residuals at $x = 0$ (clamped end with springs) and $x = L$ (free tip with mass and force):

```

1  omega_b = omega_col.detach()
2
3  # x = 0 (fixed end with spring compliance)
4  x0 = torch.zeros_like(omega_b, requires_grad=True)
5  Wr0, Wi0 = w_net(x0, omega_b)
6  E0_val = e_net(x0)
7
8  Wr0_x = grad(Wr0, x0); Wi0_x = grad(Wi0, x0)
9  Wr0_xx = grad(Wr0_x, x0); Wi0_xx = grad(Wi0_x, x0)
10 Wr0_xxx = grad(Wr0_xx, x0); Wi0_xxx = grad(Wi0_xx, x0)
11
12 # Complex EI at x=0
13 EIO_r = E0_val * I_t
14 EIO_i = E0_val * I_t * eta
15
16 # Rotational spring: E*I*W''(0) + kphi*W'(0) = 0
17 bc0_rot_r = (EIO_r*Wr0_xx - EIO_i*Wi0_xx) + kphi*Wr0_x
18 bc0_rot_i = (EIO_r*Wi0_xx + EIO_i*Wr0_xx) + kphi*Wi0_x
19 # Translational spring: E*I*W'''(0) + kx*W(0) = 0
20 bc0_tr_r = (EIO_r*Wr0_xxx - EIO_i*Wi0_xxx) + kx*Wr0
21 bc0_tr_i = (EIO_r*Wi0_xxx + EIO_i*Wr0_xxx) + kx*Wi0
22
23 # x = L (free tip with mass and force)
24 xL = torch.full_like(omega_b, L, requires_grad=True)
25 WrL, WiL = w_net(xL, omega_b)
26 EL_val = e_net(xL)
27
28 WrL_x = grad(WrL, xL); WiL_x = grad(WiL, xL)
29 WrL_xx = grad(WrL_x, xL); WiL_xx = grad(WiL_x, xL)
30 WrL_xxx = grad(WrL_xx, xL); WiL_xxx = grad(WiL_xx, xL)
31
32 EIL_r = EL_val * I_t
33 EIL_i = EL_val * I_t * eta
34
35 # Zero moment at tip: E*I*W''(L) = 0
36 bcL_m_r = EIL_r*WrL_xx - EIL_i*WiL_xx
37 bcL_m_i = EIL_r*WiL_xx + EIL_i*WrL_xx
38 # Tip mass + force balance: E*I*W'''(L) + m_tip*omega^2*W(L) + F = 0
39 # (force is purely real, so it only appears in the real residual)
40 bcL_f_r = EIL_r*WrL_xxx - EIL_i*WiL_xxx + m_tip_t*omega_b**2*WrL + F
41 bcL_f_i = EIL_r*WiL_xxx + EIL_i*WrL_xxx + m_tip_t*omega_b**2*WiL
42
43 bc_loss = torch.mean(
44     bc0_rot_r**2 + bc0_rot_i**2 +
45     bc0_tr_r**2 + bc0_tr_i**2 +
46     bcL_m_r**2 + bcL_m_i**2 +
47     bcL_f_r**2 + bcL_f_i**2
48 )

```

Listing 13: PINN loss: boundary residuals

Finally the data residual — the only term that touches the training data:

```

1  # Data loss: predicted velocity FRF vs experimental

```

```

2   Wr_d, Wi_d = w_net(x_t, omega_t)
3   Vp_r = -omega_t * Wi_d      #  $Re(j\omega W) = -\omega Im(W)$ 
4   Vp_i =  omega_t * Wr_d      #  $Im(j\omega W) = +\omega Re(W)$ 
5   data_loss = torch.mean((Vp_r - Vr_t)**2 + (Vp_i - Vi_t)**2)
6
7   total = w_pde*pde_loss + w_bc*bc_loss + w_data*data_loss
8
9   logs = {
10      "loss": total.item(),
11      "pde":  pde_loss.item(),
12      "bc":   bc_loss.item(),
13      "data": data_loss.item(),
14      "eta":  eta.item(),
15      "kx":   kx.item(),
16      "kphi": kphi.item(),
17      "F":    F.item(),
18  }
19  return total, logs

```

Listing 14: PINN loss: data fit and total

What's worth noticing about the structure:

- Every physical equation that has a complex form (PDE, BCs, FRF relation) is expanded into two real equations — one for the real part, one for the imaginary part. The pattern $(a+jb)(c+jd) = (ac - bd) + j(ad + bc)$ is applied throughout.
- BC evaluations create fresh tensors `x0` and `xL` with `requires_grad=True`. This is necessary because they're new graph leaves; you can't take `grad` with respect to a tensor that doesn't have `requires_grad` set.
- `omega_b = omega_col.detach()` prevents accidental backprop through ω , which would be physically meaningless and computationally wasteful.
- The loss returns both the differentiable tensor (for backprop) and a dict of detached scalars (for logging). This is a common pattern to keep training loops clean.

9.11 Training loop (Phase 1: Adam)

```

1  EPOCHS_ADAM = 20000
2  W_PDE, W_BC, W_DATA = 10.0, 100.0, 1.0
3
4  phys_params = {"eta_raw": eta_raw, "kx_raw": kx_raw,
5                "kphi_raw": kphi_raw, "F_raw": F_raw}
6
7  # Auto-resume from Drive checkpoint if present
8  start_epoch, saved_history = load_adam_checkpoint(
9      w_net, e_net, phys_params, optimizer, scheduler)
10
11  history = saved_history if saved_history is not None else {
12      k: [] for k in ["loss", "pde", "bc", "data", "eta", "kx", "kphi", "F"]
13  }
14
15  for epoch in range(start_epoch, EPOCHS_ADAM):
16      optimizer.zero_grad()
17      loss, logs = pinn_loss(W_PDE, W_BC, W_DATA)
18      loss.backward()
19      optimizer.step()
20      scheduler.step()          # no argument -- CosineAnnealing is epoch-based
21
22  for k in history:

```

```

23     history[k].append(logs[k])
24
25     if epoch % 1000 == 0:
26         # print progress
27         ...
28
29     if epoch % ADAM_CKPT_EVERY == 0 or epoch == EPOCHS_ADAM - 1:
30         save_adam_checkpoint(epoch, w_net, e_net, phys_params,
31                               optimizer, scheduler, history)

```

Listing 15: Adam training loop with auto-resume

The loop structure is conventional, except for two things:

- Auto-resume: `load_adam_checkpoint` returns a `start_epoch` of 0 if no checkpoint exists, otherwise the epoch to resume from. The loop just starts from that index, so no special logic is needed in the main code.
- Checkpoint after the very last epoch (`epoch == EPOCHS_ADAM - 1`) regardless of the regular cadence — this guarantees the final state is saved.

9.12 Training loop (Phase 2: L-BFGS)

```

1  all_params = (list(w_net.parameters()) +
2                list(e_net.parameters()) +
3                [eta_raw, kx_raw, kphi_raw, F_raw])
4
5  opt_lbfgs = torch.optim.LBFGS(
6      all_params, lr=0.1, max_iter=20,
7      history_size=50, line_search_fn="strong_wolfe"
8  )
9
10 LBFGS_STEPS = 300
11 start_step = load_lbfgs_checkpoint(w_net, e_net, phys_params, opt_lbfgs)
12
13 for step in range(start_step, LBFGS_STEPS):
14     def closure():
15         opt_lbfgs.zero_grad()
16         loss, _ = pinn_loss(W_PDE, W_BC, W_DATA)
17         loss.backward()
18         return loss
19
20     opt_lbfgs.step(closure)
21
22     if step % 10 == 0:
23         # log progress
24         ...
25
26     if step % LBFGS_CKPT_EVERY == 0 or step == LBFGS_STEPS - 1:
27         save_lbfgs_checkpoint(step, w_net, e_net, phys_params, opt_lbfgs)

```

Listing 16: L-BFGS refinement

L-BFGS in PyTorch uses a slightly different API: you pass a `closure` function that computes the loss and gradients, and L-BFGS may call it multiple times per `step()` (the line search needs multiple loss evaluations to find a good step length). Strong Wolfe is a robust line search that almost always finds a good step.

9.13 Checkpointing

```

1  def save_adam_checkpoint(epoch, w_net, e_net, physical_params,
2                          optimizer, scheduler, history):
3      torch.save({
4          "epoch":      epoch,
5          "w_net":      w_net.state_dict(),
6          "e_net":      e_net.state_dict(),
7          "phys":      {k: v.detach().clone() for k, v in physical_params.items
8                        ()},
9          "optimizer":  optimizer.state_dict(),
10         "scheduler":  scheduler.state_dict(),
11         "history":    history,
12     }, CKPT_ADAM)
13
14 def load_adam_checkpoint(w_net, e_net, physical_params, optimizer, scheduler):
15     if not os.path.exists(CKPT_ADAM):
16         return 0, None
17     ckpt = torch.load(CKPT_ADAM, map_location=device, weights_only=False)
18     w_net.load_state_dict(ckpt["w_net"])
19     e_net.load_state_dict(ckpt["e_net"])
20     for k, v in ckpt["phys"].items():
21         physical_params[k].data.copy_(v)
22     optimizer.load_state_dict(ckpt["optimizer"])
23     scheduler.load_state_dict(ckpt["scheduler"])
24     return ckpt["epoch"] + 1, ckpt["history"]

```

Listing 17: Checkpoint save/load functions

Important: when loading scalar parameters, use `data.copy_(v)` rather than reassigning the parameter. The reason is that the optimizer (Adam) holds references to the parameter tensors. If you replace the underlying tensor, the optimizer keeps pointing at the old one. `data.copy_` writes new values into the existing tensor in place.

9.14 Evaluation and outputs

After training, the notebook produces three artifacts:

```

1  w_net.eval(); e_net.eval()
2
3  x_plot = np.linspace(0, L, 200)
4  x_plot_t = torch.tensor(x_plot, dtype=DTYPE, device=device).view(-1, 1)
5
6  with torch.no_grad():
7      E_profile = e_net(x_plot_t).cpu().numpy().reshape(-1)

```

Listing 18: Evaluating $E(x)$ across the beam

```

1  with torch.no_grad():
2      Wr_p, Wi_p = w_net(x_eval_t, omega_eval_t)
3      Vp_r = (-omega_eval_t * Wi_p).cpu().numpy().reshape(-1)
4      Vp_i = ( omega_eval_t * Wr_p).cpu().numpy().reshape(-1)
5      V_pred = Vp_r + 1j * Vp_i
6
7  results = {
8      "eta": float(eta_f.item()),
9      "kx": float(kx_f.item()),
10     "kphi": float(kphi_f.item()),
11     "F": float(F_f.item()),
12     "E_profile_x_mm": (x_plot*1000).tolist(),
13     "E_profile_GPa": (E_profile/1e9).tolist(),
14     "geometry": {"L": L, "width": width, "height": height,

```

```

15         "rho": rho, "m_tip": m_tip},
16         "freq_range_hz": [MIN_FREQ_HZ, MAX_FREQ_HZ],
17     }
18     with open("pinn_results.json", "w") as f:
19         json.dump(results, f, indent=2)
20
21     torch.save({
22         "w_net": w_net.state_dict(),
23         "e_net": e_net.state_dict(),
24         "phys": {k: v.detach().clone() for k, v in phys_params.items()},
25         "history": history,
26     }, CKPT_DONE)

```

Listing 19: Predicting the FRF and saving results

`w_net.eval()` and `e_net.eval()` put the networks in evaluation mode. For these architectures (no dropout, no batch norm) it makes no functional difference, but it's good practice. `torch.no_grad()` disables autograd tracking for the evaluation pass — saves memory and is faster.

9.15 End-to-end flow recap

The execution sequence for a typical run, from notebook start to results:

1. Set seeds, configure dtype and device.
2. Declare geometry constants and initial guesses.
3. Upload data file; run inspection cell to verify structure.
4. Visualise raw FRF; choose frequency cutoff.
5. Run preprocessing: cutoff, detect peaks on smoothed $|V|$, build peak-aware training set from raw V .
6. Convert to tensors; define normalisation functions.
7. Construct WNet, ENet, scalar parameters.
8. Build Adam optimizer holding everything; attach cosine annealing scheduler.
9. Build collocation grid.
10. Run Adam loop (20,000 epochs), with auto-checkpointing every 1000 epochs.
11. Run L-BFGS loop (300 steps), with auto-checkpointing every 30 steps.
12. Evaluate $E(x)$ on a fine grid; produce profile plot.
13. Print identified scalars.
14. Compare predicted FRF to experiment; produce overlay plot.
15. Save JSON, CSV, and final state checkpoint.

If the Colab session disconnects partway through, the loops re-detect their checkpoints and resume without manual intervention.

10. Implementation details (lower-level)

10.1 Manual real/imaginary handling

PyTorch’s autograd doesn’t support complex numbers well, and most numerical operations on complex tensors are slower or unstable. The implementation handles complex arithmetic manually by splitting every quantity into two real tensors (`_r` and `_i` suffixes) and propagating them through the equations using algebraic identities:

$$(a + jb)(c + jd) = (ac - bd) + j(ad + bc) \quad (17)$$

```
1 result_r = a_r * c_r - a_i * c_i
2 result_i = a_r * c_i + a_i * c_r
```

Listing 20: Manual complex multiplication

This is why the loss code looks long — every line that would be a single complex operation is expanded into two real operations.

10.2 Why fourth-order autograd is the bottleneck

The PDE residual needs W'''' at every collocation point. Each `grad` call:

- Doubles the computational graph size (the gradient is itself a tensor with its own dependencies).
- Requires a full backward pass to compute.
- Is **sequential** — you can’t compute W'''' in parallel with W''' ; the chain is dependent.

So one `pinn_loss` call does the equivalent of ~ 16 standard forward-backward passes (4 sequential gradient calls $\times$ 4 components: $W_r, W_i, \text{ENet}_x, \text{ENet}_{xx}$). This is the dominant cost in each training epoch and the reason the GPU doesn’t speed things up much — parallelism helps across the batch of collocation points, not down the sequential derivative chain.

10.3 Normalisation

Inputs to both networks are normalised:

$$\text{norm_x}(x) = 2x/L - 1 \quad [0, L] \rightarrow [-1, 1] \quad (18)$$

$$\text{norm_omega}(\omega) = 2(\omega - \omega_{\min})/(\omega_{\max} - \omega_{\min}) - 1 \quad [\omega_{\min}, \omega_{\max}] \rightarrow [-1, 1] \quad (19)$$

This keeps tanh-activated layers in their well-conditioned regime. Without normalisation, the network has to learn to compensate for large input scales, which slows training.

11. Outputs and validation

11.1 Files produced

`pinn_results.json`

Contains the identified scalars and the spatial profile of $E(x)$:

```
1 {
2   "eta":    0.012,
3   "kx":    8.2e5,
4   "kphi":   3.4e3,
```



```

5  "F":      0.083,
6  "E_profile_x_mm": [0, 0.63, 1.27, ...],
7  "E_profile_GPa":  [195.2, 198.7, 201.1, ...],
8  "geometry": {...},
9  "freq_range_hz": [1.0, 5000.0]
10 }

```

Listing 21: Example pinn_results.json

pinn_frf_comparison.csv

The PINN’s predicted FRF at the training frequencies, alongside the experimental values. Useful for plotting overlays in external tools.

11.2 Plots produced during the notebook run

1. **Identified $E(x)$:** $E(x)$ evaluated at 200 uniformly-spaced positions along the beam, plotted vs position in mm. Look for plausible mean level (close to your initial guess) and smooth variation. Wild oscillations or hitting the sigmoid bounds (50 or 400 GPa) indicate trouble.
2. **PINN vs Experimental FRF:** predicted velocity FRF overlaid on measured. Should match peak *positions* (driven by $E(x)$ and BCs) and peak *heights* (driven by η and F). Mismatches diagnose specific failures:
 - Wrong peak positions $\rightarrow$ check $E(x)$ mean level / boundary springs.
 - Right positions, wrong heights $\rightarrow$ check η or F convergence.
 - Flat prediction $\rightarrow$ training didn’t converge; check loss history.
3. **Loss history (PDE / BC / data):** all three should decrease over training; if one stays high while others drop, the weights need rebalancing.

11.3 What “converged” looks like

Healthy convergence has these properties:

- All three loss terms decrease monotonically (allowing for restart spikes from cosine annealing).
- η ends up in 0.001–0.05 for typical metals.
- k_x finishes far from its initial value (10^5) — typically 10^4 to 10^7 depending on clamp quality.
- k_φ finishes in 10^2 to 10^5 N·m/rad.
- F finishes in 0.01 to 1 N.
- $E(x)$ mean is within $\pm 50\%$ of the assumed material.
- The predicted FRF visually matches the experimental peaks.

11.4 When to suspect a problem

- Loss plateaus very high ($> 10^{-2}$ normalised) $\rightarrow$ physics not being satisfied; try rebalancing weights.
- Loss spikes periodically $\rightarrow$ LR scheduler problem (you should be on cosine annealing, not ReduceLROnPlateau).

- η pegged at zero or $> 0.5 \rightarrow$ either preprocessing destroyed peak heights or the network can't fit them; check the preprocessing diagnostic plot.
- $E(x)$ hits the sigmoid bounds $\rightarrow$ either the bounds are wrong for your material or the optimizer is exploring nonphysically; widen bounds or check initial guess.

12. Open issues and next steps

12.1 Synthetic validation (supervisor concern)

Currently the pipeline trains directly on experimental data without first verifying on a synthetic problem where ground-truth parameters are known. Standard practice in PINN papers includes a synthetic benchmark:

1. Pick an $E(x), \eta, k_x, k_\varphi, F$ (the “ground truth”).
2. Generate a synthetic FRF analytically (modal expansion, finite element method, etc.).
3. Add realistic noise.
4. Run the full pipeline on this synthetic data.
5. Confirm it recovers parameters close to the ground truth.

This needs to be incorporated before the experimental results can be defended rigorously.

12.2 Coherence- or SNR-weighted data loss (fine-tuning step)

Different points in the FRF have different measurement reliability. Near peaks, SNR is high; in noise-floor valleys, SNR is low. Weighting the data loss by per-point reliability would let the optimizer ignore unreliable regions:

$$L_{\text{data}} = \sum_i w_i \cdot |V_{\text{pred}}(\omega_i) - V_{\text{meas}}(\omega_i)|^2 \quad (20)$$

where w_i is high near peaks, low in valleys. Two ways to get the weights:

- **Coherence function** $\gamma^2(f)$ from the analyser (requires raw input/output spectra, not just the FRF).
- **SNR estimate** $|V|^2/(|V|^2 + \sigma^2)$ with σ estimated from a noise-only band.

Best added *after* the basic v6 pipeline is working — easier to attribute improvements that way.

12.3 Multi-experiment warm-starting

Currently each experiment retrains from scratch (~ 1.5 hours on a T4). For experiments with the same material, a saved checkpoint could be reused as a warm-start. Per-experiment retraining time could drop from ~ 1.5 h to ~ 20 minutes.

12.4 HPC for parallel experiment campaigns

GPU parallelism doesn't accelerate single PINN runs much (the fourth-order autograd chain is sequential). But running multiple experiments in parallel on an A100/H100 cluster is straightforward and scales linearly. If you need to process a dozen specimens, that's where HPC pays off — not for faster individual runs.

12.5 Phase loss term

Currently the data loss compares only $\text{Re}(V)$ and $\text{Im}(V)$. An explicit phase constraint at detected resonance frequencies (e.g., penalise $\text{Im}(V) \neq 0$ at peak ω values, since at velocity-FRF resonances $\text{Im}(V) = 0$ exactly) could help lock in correct mode positions. Scoped but not yet implemented.

A. Notebook cell map

For navigation, here's where each piece of the pipeline lives in `pinn_v5_final__current__.ipynb`:

Cell	Content
1	Imports, device setup, checkpoint helpers
3	Beam geometry constants, frequency range, initial parameter guesses
5	<code>load_frf</code> — reads <code>.npz</code> , normalises structure, handles duplicates
7	File inspection — prints structure of uploaded file
9	Raw FRF visualisation (magnitude, phase, Re/Im)
11	Preprocessing — cutoff, smoothing, peak detection, training point selection
13	Tensor conversion, normalisation functions
15	WNet and ENet definitions
17	Scalar parameter setup, Adam optimizer, LR scheduler
19	Collocation grid construction
21	<code>grad</code> helper
23	<code>pinn_loss</code> — full loss function with PDE, BC, data terms
25	Adam training loop (Phase 1, 20k epochs)
27	L-BFGS training loop (Phase 2, 300 steps)
29	Loss history plotting
31	$E(x)$ profile plot
33	Final scalar parameter printout
35	PINN vs experimental FRF comparison plot
37	Save results (JSON, CSV, final checkpoint)
38	Utility cells (fresh start, reload)